

TUGAS 1

Login dengan Password Terenkripsi

Laravel 13

Tujuan	Mahasiswa mampu membuat sistem login yang aman dengan Laravel, mencakup password ter-hash, proteksi halaman, dan logout.
Prasyarat	PHP 8.3 atau lebih tinggi, Composer, dan MySQL (XAMPP/Laragon). Laravel 13 dirilis pada 17 Maret 2026 dan membutuhkan PHP 8.3 atau lebih tinggi.

Tampilan Acuan (Mockup)

1. /login (contoh saat login gagal)

```
+-----+
| localhost:8000/login |
+-----+
| Login |
| Masuk untuk membuka dashboard |
| |
| [ Username atau password salah. ] |
| |
| Username |
| [ budi ] |
| Password |
| [ ***** ] |
| |
| [          Masuk          ] |
+-----+
```

2. /dashboard (setelah login berhasil)

```
+-----+
| localhost:8000/dashboard |
+-----+
| Dashboard [ Logout ] |
+-----+
| Selamat datang, Budi Santoso! |
| Halaman ini hanya bisa dibuka |
| setelah login. |
| Username: budi |
+-----+
```

3. /dashboard dibuka tanpa login

```
localhost:8000/dashboard
|
|   pengguna belum login
v
otomatis dialihkan ke localhost:8000/login
```

Ketentuan

1. Buat proyek dengan `composer create-project laravel/laravel:^13.0 tugas1`.
Dilarang memakai starter kit (Breeze, Jetstream, dan sejenisnya); logika login ditulis sendiri.
2. Atur `.env` agar memakai MySQL (`DB_CONNECTION=mysql`, `DB_DATABASE=db_tugas`, dan seterusnya). Buat database kosong `db_tugas`.
3. Tabel `users` dibuat lewat **migration** dengan kolom: `id`, `username` (unik), `password`, `nama_lengkap`, `timestamps`. Ubah migration bawaan agar memakai `username`, bukan `email`.
4. Isi minimal 2 user lewat **seeder** (`php artisan db:seed`). Password disimpan dalam bentuk hash. Pastikan model `User` menyesuaikan kolom baru (`$fillable`) dan memakai cast `'password' => 'hashed'`.
5. Buat `AuthController` dan route berikut:
 - `GET /login`: menampilkan form (route diberi nama `login`).
 - `POST /login`: memvalidasi input, lalu memakai `Auth::attempt()`.
 - Jika berhasil: `$request->session()->regenerate()` dan arahkan ke `/dashboard`.
 - Jika gagal: kembali ke form dengan pesan "Username atau password salah." tanpa menyebut mana yang salah.
 - `GET /dashboard`: menampilkan "Selamat datang, [nama_lengkap]!" dan tombol **Logout**.
 - `POST /logout`: memanggil `Auth::logout()`, `session()->invalidate()`, `session()->regenerateToken()`, lalu mengarahkan ke `/login`.
6. Proteksi route:
 - `/dashboard` dan `/logout` memakai middleware `auth`. Dashboard yang dibuka langsung lewat URL tanpa login otomatis dialihkan ke `/login`.
 - `/login` memakai middleware `guest`. Pengguna yang sudah login langsung diarahkan ke dashboard.
7. Tampilan memakai **Blade** (`login.blade.php`, `dashboard.blade.php`).

Pertanyaan (maksimal 1 halaman)

- a. Mengapa password harus di-hash? Lihat kolom `password` di tabel `users` dan jelaskan isinya.
- b. Mengapa logout memakai metode POST dengan `@csrf`, bukan link biasa?
- c. Apa fungsi middleware `auth` dan `guest`? Apa yang terjadi jika `/dashboard` dibuka tanpa login?
- d. Apa fungsi `session() -> regenerate()` setelah login berhasil?

- a. Password harus di-hash untuk menjaga keamanan akun pengguna. Hashing mengubah password menjadi rangkaian karakter yang sulit dibaca dan tidak menyimpan password asli secara langsung di database.

Pada kolom `password` di tabel `users`, isinya berupa teks hash, bukan password asli yang dimasukkan pengguna. Contohnya, password `tAehyungi3!` akan disimpan dalam bentuk hash yang panjang dan acak. Saat login, Laravel memeriksa kecocokan password yang dimasukkan dengan hash tersebut menggunakan proses verifikasi, tanpa perlu menyimpan password asli.

- b. Logout menggunakan metode POST dengan `@csrf` karena logout merupakan aksi yang mengubah status autentikasi pengguna, yaitu mengakhiri sesi login. Metode POST lebih tepat daripada link biasa yang menggunakan GET karena GET seharusnya digunakan untuk mengambil atau menampilkan data, bukan menjalankan aksi yang mengubah keadaan aplikasi.

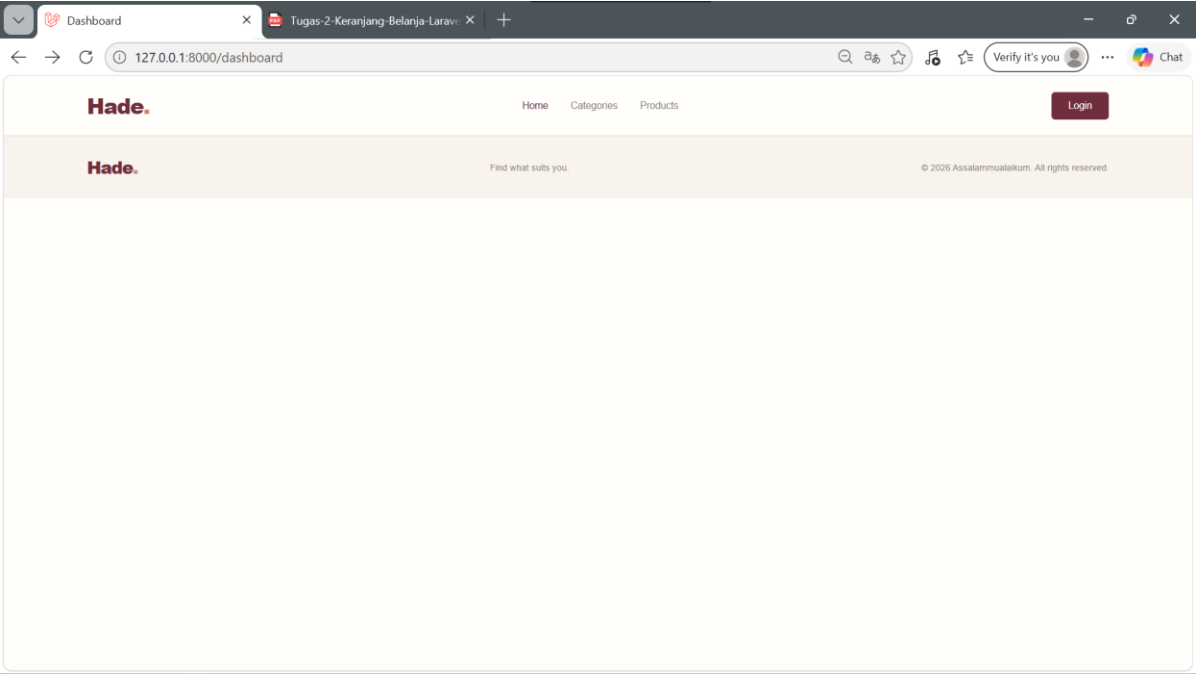
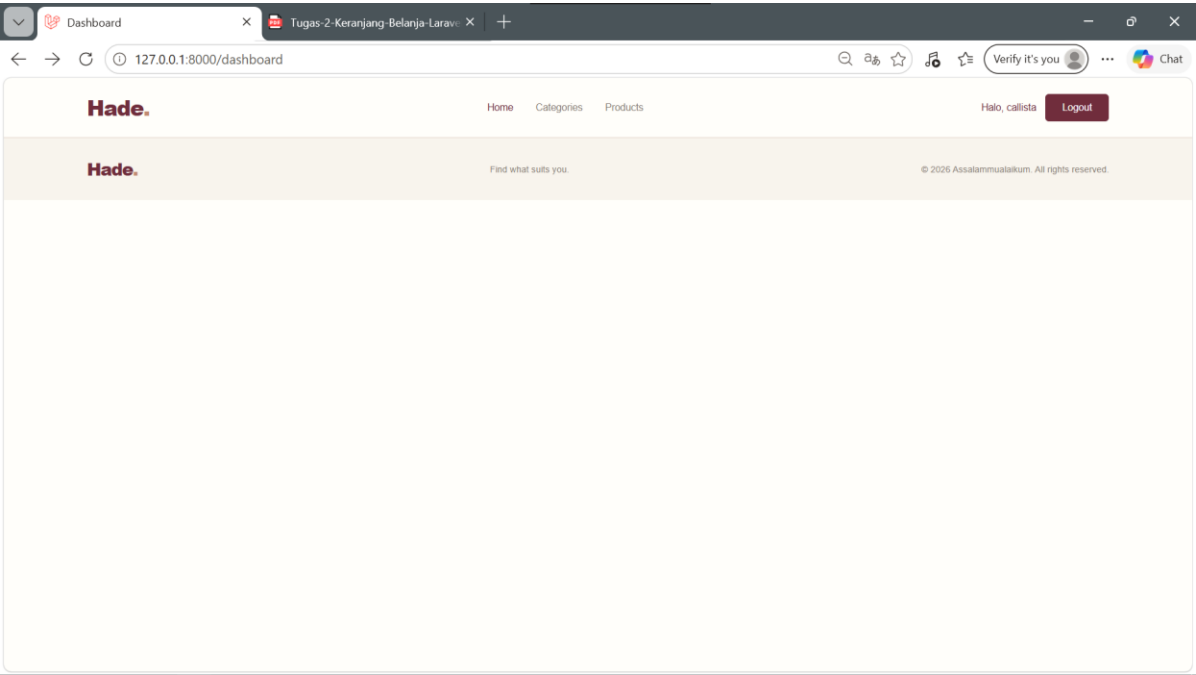
Sementara itu, `@csrf` digunakan untuk melindungi aplikasi dari serangan Cross-Site Request Forgery (CSRF), yaitu ketika pihak lain mencoba membuat pengguna menjalankan aksi tanpa disadari. Dengan demikian, proses logout menjadi lebih aman dan sesuai dengan praktik keamanan Laravel.

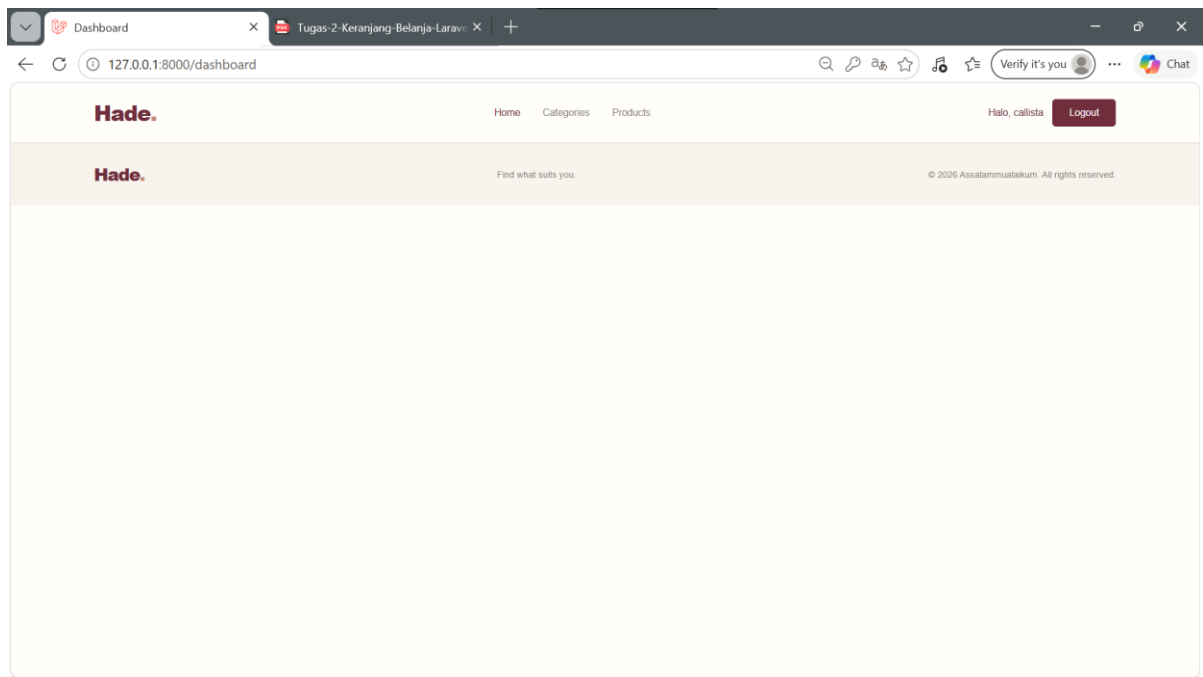
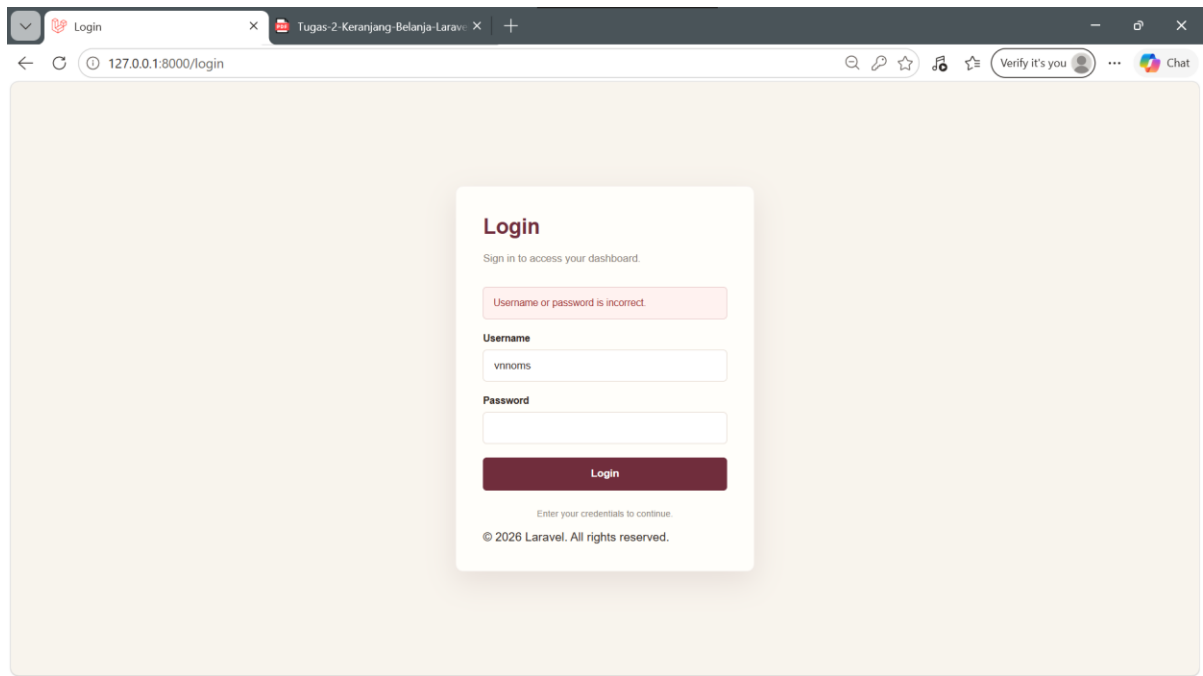
- c. Middleware `auth` berfungsi untuk membatasi akses halaman hanya bagi pengguna yang sudah login, sedangkan middleware `guest` membatasi halaman agar hanya bisa diakses oleh pengguna yang belum login, seperti halaman login.

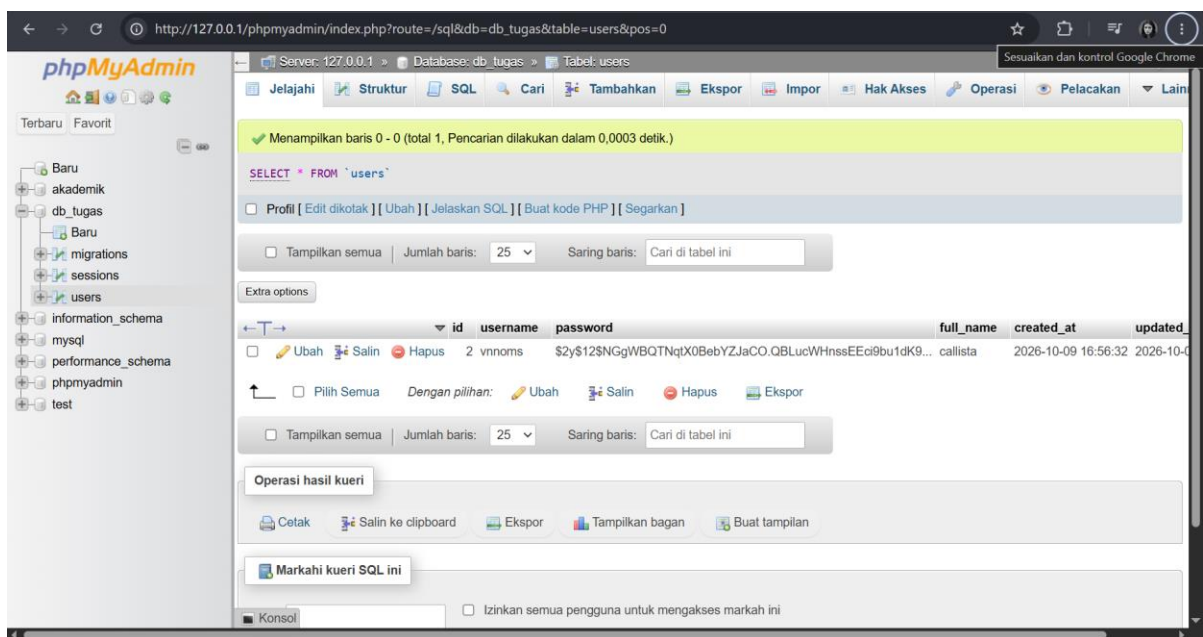
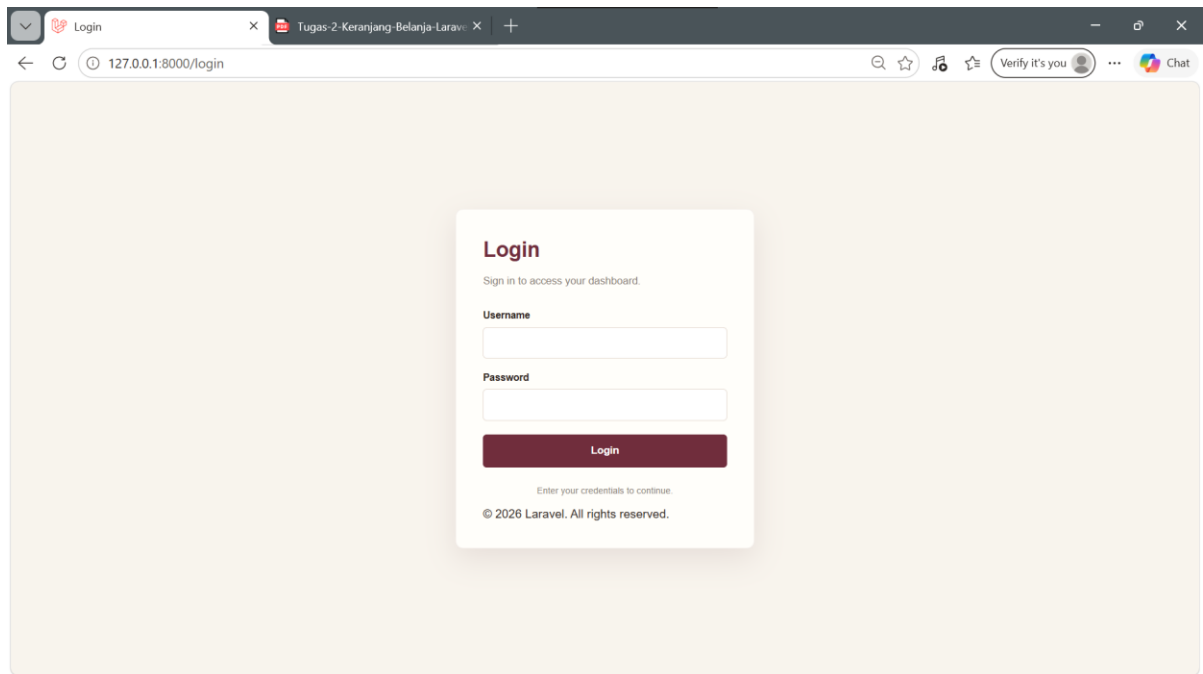
Pada proyek ini, `/dashboard` bisa dibuka tanpa login karena route dashboard tidak menggunakan middleware `auth`. Jika middleware `auth` diterapkan, pengguna yang belum login akan diarahkan ke halaman login.

- d. `session()->regenerate()` berfungsi untuk membuat ID session baru setelah login berhasil. Tujuannya adalah mencegah session fixation, yaitu serangan ketika pihak lain mencoba memanfaatkan ID session yang sudah diketahui sebelumnya. Dengan begitu, keamanan sesi pengguna meningkat setelah berhasil login.

Lampiran







Yang Dikumpulkan

- Folder proyek.
- Screenshot:
- isi tabel `users` (password berupa hash)
- login gagal
- login berhasil
- dashboard

- `/dashboard` dibuka di jendela incognito tanpa login
- setelah logout
- Jawaban pertanyaan.